1 What I built and why

My main decision was to admit work only after checking estimated capacity. A plain Python async function retrieves page-bound BM25 passages from three fixed PDFs, asks the model for a cited draft, then runs either a citation-ID check or a second model check. A bounded queue, minute gate, deadline, and daily guard limit work. The halved-limit run shows that local estimates cannot guarantee completion when provider limits differ.

2 The real service and my stand-in

I probed `qwen/qwen3.8-27b` on Groq's free tier. The stand-in streams chunks, adds load-sensitive latency, simulates failures and hangs, and enforces separate request, total-token and input-token limits. Its assumed 2.5% hang chance comes from one 30-second stall among 42 mixed probes; this is a weak calibration, not a reliable provider failure rate.

Table 1. Real service limits

Limit	Value found	How found	Confidence
Requests/min	30	30 short successes, then 429	High at probe time
Tokens/min	8,000 total; 7,000 input	Header; input-limit 429 body	High; medium
Requests/day	1,000	Response header and Limits page	High at probe time
Tokens/day	200,000	Limits page; not exhausted	Medium
Requests in flight	At least 2; maximum not measured	Two overlapping probe calls	Low
Limit response	HTTP 429, `rate_limit_exceeded`, `retry-after` 2s or 20s	Saved headers and bodies	High for two events
Recovery time	Not measured; a later call succeeded after a 54s wait	RPM follow-up	Low

3 Results

Selected saved run IDs: real steady `7659dad0ae`; stand-in steady `865f6788bb`, burst `776291cd23`, overload `d492c0c3f8`, halved `2d4699a7de`, exhausted `987c33eec9`. Stand-in steady is **5x time-compressed**: arrivals, refill, latency and deadline share the scaled clock. Percentiles use delivered answers (full or excerpt), linear interpolation, with timeouts separate; virtual times are rescaled. The 2.29s deadline is an assumption from a few probe latencies. Real steady has only nine delivered samples, so p95/p99 are not reported.

Table 2a. Latency

Scenario	Source	Sent	Duration s	Offered/min	First token p50/p95/p99 ms	Total p50/p95/p99 ms
Steady, 20s	Real	10	400	1.5	2,096 / not reported / not reported	2,096 / not reported / not reported
Steady, 2.29s	Stand-in, 5x	100	4,000	1.5	424.5 / 510.3 / 517.1	424.5 / 510.3 / 517.1
Burst, 2.29s	Stand-in	100	5	1,200	0.8 / 497.8 / 740.2	0.8 / 497.8 / 740.2
Overload, 2.29s	Stand-in	100	35	171.4	0.7 / 1.0 / 67.2	0.7 / 1.0 / 67.2
Minute halved, 20s	Stand-in	40	24	100	0.7 / 1.3 / 445.2	0.7 / 1.3 / 445.2
Daily exhausted, 20s	Stand-in	10	5	120	not measured	not measured

Table 2b. Outcomes (all arrivals as denominator)

Scenario	Succeeded %	Rate-limited %	Timed out/given up %	Retries/request	Peak queue	Tokens/answer	Cost/1,000 answers
Steady real	90.0 (9/10)	0.0	10.0 (1/10)	0.00	1	1,496	$0.00
Steady stand-in	99.0 (99/100)	0.0	1.0 (1/100)	0.00	1	1,778	$0.00
Burst	99.0 (99/100)	0.0	1.0 (1/100)	0.00	13	30	$0.00
Overload	99.0 (99/100)	0.0	1.0 (1/100)	0.00	1	30	$0.00
Minute halved	97.5 (39/40)	0.0	2.5 (1/40)	0.00	1	76	$0.00
Daily exhausted	0.0 (0/10)	100.0 (10/10)	0.0	0.00	1	not measured	not measured

"Succeeded" includes **98 labelled excerpts** in each burst and overload run, and **38 excerpts** in the halved run; only one full answer was delivered in each. Free-tier cash cost excludes compute and quota opportunity cost. Ten real steady requests delivered nine; one hit a local `ConnectTimeout` and never reached Groq.

4 Where the time goes

Probe call 7 had no queue wait and hit `ReadTimeout` after 30,171 ms without response headers or a first token. Groq logged `499 Client Disconnected`, 30.000 s latency, and zero tokens. The saved `work_ms` is time inside the call, not proof of generation. Real steady request 8 hit the 5-second `ConnectTimeout` cap and never reached Groq; the cause of the outside-Groq gap in delivered requests is not isolated. Nine paired client-minus-console times had median 1,765 ms and range 454–6,220 ms, assuming chronological pairing.

5 When a limit is hit

The client waits only while its deadline permits. It retries once after a 429 with `retry-after`, never after a timeout. In two halved-limit requests, roughly 12 seconds of retry-after sleep plus eight seconds of local pacing exhausted the 20-second deadline without a second HTTP attempt. Pressure fallback returns a labelled retrieved excerpt; a limit notice has `answer_kind=none` and is never counted as an answer.

Retrieval: quality versus cost

The same 15 frozen questions ran on all three settings with deterministic verification. Grades compare saved answers with cited PDF pages; `refused_wrongly` counts as wrong. The four-question model-verifier subset was chosen **after** viewing results: mean tokens rose from 1,422 to 2,830 and median total time from 711 to 1,540ms. This measures cost, not quality.

Table 3. Retrieval settings

Setting (words x pieces; rerank)	Questions	Correct+cited %	Wrong %	Correctly refused %	Tokens/answer	First p50/p95 ms	Total p50/p95 ms
Small (300 x 3; no)	15	60.0 (9/15)	20.0 (3/15)	20.0 (3/15)	1,528	563 / 764	563 / 764
Medium (450 x 4; no)	15	66.7 (10/15)	13.3 (2/15)	20.0 (3/15)	2,653	734 / 1,697	734 / 1,697
Large (700 x 4; no)	15	66.7 (10/15)	13.3 (2/15)	20.0 (3/15)	3,446	698 / 1,067	698 / 1,067

I choose small for the quota-constrained path: medium and large each recovered M03's missing fact but used 74% and 126% more tokens. Both still missed H02 and H03, so extra context did not fix the central errors. Small's M03 answer was incomplete, a real quality cost; no setting is proven best by 15 questions. Near a limit, retrieval stays local and the model call becomes a labelled excerpt or notice.

How this library will mislead a naive retriever

H02: "How many bytes per chunk?" The H-Net p.3 passage reporting 4.5-5 bytes ranked only 6th/9th, while other uses of "token" crowded it out. **H03:** "What does smoothing apply, and why?" The EMA and gradient-flow explanation spans PDF pp.6-7; a p.3 caption or p.18 ablation looks relevant but misses the mechanism. **T03:** "Why positional encodings?" A Transformer caption-first chunk can match the term before the p.6 explanation that no recurrence or convolution encodes order.

6 Retries

In the saved seed-7 stand-in experiment with a lower server RPM, zero retry made 40 HTTP calls and delivered nine answers. Unbounded no-jitter retry made 71 calls and delivered ten, but p95 delivery for the baseline's first-attempt-success cohort rose from 360ms to 6,391ms; one such request lost its answer. The criterion was written before this run. Jitter did not remove quota contention in this seed.

7 What the person sees

Under minute pressure: “The model is busy at its current limit. Please retry shortly.” At daily exhaustion: “The daily model budget is exhausted. I cannot check the papers right now. Please try after the quota resets.” Neither is graded as an answer.

8 What I'm unsure about, what I left out, what I learned

The least certain assumption is the stand-in hang probability: one stall cannot estimate a stable tail rate. If wrong, stand-in timeout percentages will not transfer. I deliberately left out embeddings/reranking and a persistent multi-process queue; neither was necessary to expose the measured bottlenecks within the deadline. I learned that a retry can consume a deadline without making a second HTTP attempt, and retries can take capacity from first attempts.

Your feedback on Vaani

The prompt-diff review before accepting an agent change and the Memories feature stood out on vaani.dev. Two different start destinations made the entry path unclear. I would use one canonical start path and show a short, measured example of how prompt changes affect a live call.

How I worked with the assistant

The accompanying `prompts_from_chat.md` contains unedited user prompts extracted from this chat; the app did not provide a complete transcript export. Three prompts shaped the work: "Do NOT write code yet ... Simulate before building" made measurements precede implementation; "Make the verifier switchable ... Drop FastAPI for the main pipeline" exposed verifier cost and kept the core readable; "Before any real Groq call: add a budget guard" made shared quota an enforced constraint. I overruled the earlier FastAPI main-pipeline design. The first model-verifier run used a 2.29s load deadline and cut off three checks. I caught the mismatch in saved rows and the console log, then required a quality-run rerun with pacing outside latency and a 30s cap per call. Another assistant analysis treated 98 rate-limited rows as lost answers; inspection showed they had returned excerpts, so I required an `answer_kind` field and corrected the success definition. The experiment gates, question/page sign-off, console-usage reconciliation, and predeclared retry criterion came from my direction; the assistant implemented and analyzed them. I waited for the real large run before giving it a quality score. The saved answer grades were produced with assistant help against the PDFs and remain reviewable in `grades.csv`.